

An Overview of Techniques for Information Extraction and Sentiment Analysis

Jonathan Rueffer

Contents

1	Introduction	2
2	Information Extraction	2
2.1	The Information Extraction System Pipeline	3
2.2	LLM Techniques for IE	5
2.2.1	Data Augmentation	6
2.2.2	Prompt Design	6
2.2.3	Supervised Fine-tuning	7
3	Sentiment Analysis	7
3.1	Lexicon-based Models	8
3.2	Machine Learning Models	9
3.3	LLM-based Models	10
3.4	Sentiment Analysis in Social Media	10
4	My Project	11
4.1	Overview	11
4.2	Implementation	12
4.3	Future Work	13
5	Conclusion	13

1 Introduction

Natural Language Processing (NLP) is a major subfield of Artificial Intelligence (AI) that continues to see much attention in current research and development. NLP focuses on enabling computers to understand and interpret human language, requiring sophisticated methods that can account for all the nuances associated with a language’s structure. One such challenge is the handling of *homographs*—words that are spelled identically but have different meanings. Accurately and efficiently addressing the more unstructured, ambiguous aspects of human language is at the forefront of current NLP developments [4].

This paper focuses on two specific applications of NLP techniques: Information Extraction and Sentiment Analysis. Information Extraction (IE) is the process of identifying and structuring specific pieces of information from unstructured, human language-formatted text [8]. Sentiment Analysis (SA) refers to the process of detecting and classifying the emotional undertone of a given text. For example, this might involve determining if an online product review is positive or negative [3]. Following a thorough exploration of both NLP applications and the methodologies commonly used to implement them, the paper concludes with a discussion of a personal project containing ER and SA features.

2 Information Extraction

Real-world textual data produced by humans—such as social media post, email, and articles—are unstructured. These texts vary in length, style, and content, and do not follow a pre-defined structure or model. The lack of standardization makes it difficult for computers to interpret and organize this type of data. Information Extraction (IE) aims to convert raw text into a machine-readable format, making it easier to analyze and retrieve relevant information.

There are four main IE tasks that will be discussed in this section: Named Entity Recognition (NER), Relation Extraction (RE), Event Extraction (EE), and Topic Modeling [8]. First, NER deals with identifying proper names of people, locations, organizations, etc. RE involves determining relationships between entities, like if a specific person works at a

specific company. EE detects events mentioned in texts, such as conferences, concerts, or holidays. Topic Modeling determines the main topics or themes present within a document. The different extractable elements are crucial in enabling deeper textual analysis, including Sentiment Analysis and automatic text summarization [8].

In recent years, generative Large Language Models (LLMs) have demonstrated remarkable capabilities in text comprehension and generation and are being used to improve IE tasks. Hence, this section also explores how LLMs contribute to the advancement of the field.

2.1 The Information Extraction System Pipeline

The goal of an IE system is to accomplish IE tasks, i.e. extract entities, recognize relationships between them, and detect key events within a given text. This section outlines the steps of a typical machine learning-based IE pipeline, with each stage of the pipeline building upon the previous one, progressively transforming raw text into structured data.

The pipeline begins with a tokenizer that tokenizes the raw and unstructured document. This is a fundamental preprocessing step in the general field of NLP, enabling models to efficiently process text data in a structured way. Tokenization involves breaking down text into smaller tokens such as sentences, words, subwords (like prefixes and suffixes), or even individual characters, depending on the method used. The preprocessing step in NLP also often involves Part-of-Speech (POS) Tagging, which labels words as nouns, verbs, adjectives, etc. By highlighting the roles of words, a model can accurately understand the structure and meaning of sentences [2].

Once a document has been preprocessed, the next step in the IE system pipeline is trainable name tagging—the process of assigning name classes (e.g. person, location, organization) to words in a sentence. It is important to note that this step is the Named Entity Recognition (NER) task. A variety of learning algorithms have been designed for this task, including Hidden Markov Models (HMMs), Maximum Entropy Models, Decision Trees, and Support Vector Machines [2].

Bolt Beranek and Newman Inc. (BBN), a research and development company, introduced the NYMBLE name tagging system [2], which improved HMM-based systems. Al-

though NYMBLE has been surpassed by modern neural approaches, it remains relevant for understanding the foundations of statistical NER approaches.

Tagging systems assign classes based on probability distributions, and NYMBLE’s success stems from its ability to increase the accuracy of the generated distributions. Instead of handling words independently like HMMs, NYMBLE considers word sequences when tagging, so word pairs like “New” and “York” are not treated separately. In addition, NYMBLE processes each word sequentially and can account for properties of words like capitalization [2], a property useful for distinguishing between “Apple” as a company name and “apple” as a fruit.

To estimate probabilities more accurately, NYMBLE utilizes several levels of *back-off models* [2] that incorporate rules like the following:

- If a specific word has been frequently classified as a person, prioritize that probability when making predictions.
- If no useful word features exist, back off to class-level probabilities. For instance, proper nouns that appear in the subject position of a sentence tend to represent a person.

After the name tagging step, where entities such as persons, locations, and organizations are identified and labeled, a coreference resolver determines whether two mentions refer to the same entity [2]. Here, a mention refers to any occurrence of an entity (e.g., a person or location) in the text. So, an example of a coreference resolver’s task is to identify whether the mentions “New York” and “the city” refer to the same entity.

This task is usually converted into a supervised binary classification problem to determine whether a candidate mention is referring to a previous mention or cluster of mentions. Multiple lexical and syntactic attributes are used for this classification. For example, the existence of determiners (e.g., “the,” “a,” “some”) or quantifiers (e.g., “many,” “several,” “few”) can indicate whether a mention refers to a specific entity or a general concept. Additionally, coreference resolution often relies on the degree of string similarity—how closely two phrases resemble each other—and distance-based measures, such as the number of words or sentences separating mentions, to resolve coreferences [2].

Once the coreference resolving step is complete, IE task-specific methods can be applied: relational taggers (for RE) and event detection (for EE). For relational tagging, more generic yet well-researched algorithms are used, including K-Nearest-Neighbor and Support Vector Machines [2]. The typical features used to classify relations include the head of a mention, which is the most important word that determines its meaning (e.g. in "The College of Wooster," the head is "College"), as well as the head's class that was categorized in the name tagging step of the pipeline. The head of a mention is taken into account because a person is more likely to have a "works-for" relationship with an organization than with a location. The features used for relational tagging also include context words—words surrounding heads that provide additional information about the nature of a relationship [2].

Event detection aims to recognize specific events in texts by identifying keywords (triggers) and their associated arguments [2]. Event detection models determine if a word is a trigger using the probability computed from a training dataset. For example, the word "launched" is a trigger for a "product release" event. One way to determine trigger words is via a simple rule-based classification: if the trigger word is in a predefined event lexicon, the corresponding event type is assigned. A more common approach is through supervised learning, which trains a classifier using labeled datasets[2]. Alternatively, a pre-trained transformer model can be used to fine-tune existing models trained on large event datasets (like GPT). For a given trigger and a mention, an event detection model must also determine whether the mention is an argument of the trigger or not (e.g. is Alice the argument of a "job change" event?) and classify the argument in an event-specific role.

2.2 LLM Techniques for IE

In this section, we will look at the various techniques currently being used to adapt the strengths of LLMs to IE. These include Data Augmentation—enhancing information by applying various transformations to the existing data using LLMs; Prompt Design—using task-specific prompts to direct the behavior of a model; and Supervised Fine-tuning—training LLMs on IE tasks using labeled data.

2.2.1 Data Augmentation

LLMs are very powerful and efficient in text generation, which is why researchers leverage this strength to create synthetic textual data that can be used to improve IE models, reducing the reliance on expensive and time-consuming human annotation. Moreover, the generated data are meaningful and diverse to effectively enhance the text examples used in training while avoiding introducing unrealistic and misleading patterns. The method LL-MaAA [8], published in 2023, involves using LLMs as annotators to ensure that only the most useful examples are annotated, reducing data redundancy and in turn improving overall data efficiency.

Another way that LLMs are used for IE is with Knowledge Retrieval. This approach uses LLMs as an additional knowledge base that can be used. The retrieved auxiliary knowledge is incorporated into the model for more accurate entity predictions in NER tasks [2]. This approach is particularly useful in low-resource scenarios where labeled training data is scarce. PGIM [2] is a two-stage framework presented in 2023 for multimodal NER (multi referring to using both text and images for IE), which leveraged ChatGPT as the additional knowledge source to provide additional context when identifying entities.

A third way that LLMs are used for Data Augmentation is through Inverse Generation. This strategy utilizes the LLM training paradigm to generate realistic unstructured text given structured data as input. Current developments in this area seek to improve text diversity while maintaining text coherence and entity preservation. This method allows for high-quality synthetic data generation, improving the training process and model performance [2].

2.2.2 Prompt Design

One way to increase the capabilities of an LLM solving IE tasks is through strategically choosing and tuning prompts to improve the output. This technique is known as Prompt Design and plays an important role in fitting the LLM architecture to the structure required in IE tasks. LLM instructions are based on a dialogue-based method, a paradigm different from the predictive nature of IE tasks. This is why current improvements employ a Question

Answer (QA) prompt approach to facilitate the generation of desired results [2]. For example, in 2023 the QA4RE framework [2] introduced reformulating Relation Extraction tasks as multiple-choice questions to exploit the higher prevalence of QA tasks in the datasets used in LLM training.

A different prompting strategy often used to enhance LLM performance is Chain-of-Thought (CoT) [2], a strategy that includes a step-wise and coherent reasoning chain as part of the prompt to guide a model’s response generation. For NER tasks, CoT prompting might involve providing entity definitions and asking the LLM to reason through new entity generation. Since there is limited control over the actual model parameters, CoT provides additional fine-tunable control over how the LLM approaches a specific IE task.

2.2.3 Supervised Fine-tuning

Supervised Fine-tuning adapts pre-trained LLMs to specific IE tasks by training them further on labeled datasets, allowing the model to capture the underlying structural patterns in the data and effectively generalize to unseen samples. This strategy explicitly teaches the model how to learn the task by comparing the model output to the ground truths (given in the labels) via a loss function, which is used to optimize the model parameters accordingly. Supervised Fine-tuning is a common approach to incorporating LLMs in IE and has been demonstrated to be a successful one. For example, the DEEPSTRUCT method [2] introduced pre-training on a collection of task-agnostic textual data to increase a model’s understanding of inherent linguistic structures and patterns. The model can then utilize its deeper understanding of linguistic patterns to make more accurate predictions in various IE tasks.

3 Sentiment Analysis

Sentiment Analysis (SA) is a Natural Language Processing (NLP) task that aims to classify the emotional tone behind a piece of text. Sentiment classifications can include the binary positive/negative classes or more nuanced emotions like joy, sadness, or sarcasm. SA has far-reaching applications in various academic and industry fields. For example, businesses use

Sentiment Analysis to analyze customer reviews to make informed and profitable business decisions. It is also used across social media platforms to generate real-time user opinions that governments and organizations use to track public sentiment about political or social issues [6].

There are three main levels where SA can be applied: the document, sentence, and entity level [3]. At the document level, the whole document is taken into consideration to figure out the overall sentiment. This approach relies on the assumption that the entire document only contains one main sentiment, like a news article or product review. In turn, the main drawback of this level is that more sentimentally complex documents cannot be adequately classified.

The sentence-level approach determines the sentiment of individual sentences rather than assessing the entire document. This is useful for understanding user emotions in real-time conversations. For instance, chatbots use sentence-level Sentiment Analysis to respond more appropriately when users express varying emotions throughout an exchange. Another classification that occurs at this level is determining if individual sentences are either objective (a fact) or subjective (an opinion) [3].

Entity-level (or aspect-based) Sentiment Analysis identifies sentiments about specific entities or aspects mentioned in a sentence. This involves identifying the target entity in the text (like a product or idea), extracting the related sentiment expressions, and classifying the overall sentiment related to the entity. This is useful in many cases, like when looking at a detailed product review that could contain different sentiments about separate components of a single product [3].

The rest of this section explores the main methods used to approach this IE task: Lexicon-based Models, Machine Learning Models, and LLM-based Models. This is followed by an examination of SA in social media, as the nature of the medium poses nuanced challenges to the field.

3.1 Lexicon-based Models

Lexicon-based models use predefined sentiment word lists to identify sentiment-bearing words and aggregate their scores to determine an overall sentiment at the document level. When

analyzing a given text, these models separate words and assign them sentiment scores based on their presence in the lexicon. Then, the category with the highest score determines the final sentiment classification [6]. One way this is done is with a dictionary-based approach, where the method starts with a manually collected list of words and expands the lexicon by searching for synonyms and antonyms in dictionaries or thesauruses [6]. Other methods for lexicon-based models rely on statistical techniques to classify keywords as indicating a specific sentiment. One disadvantage to lexicon-based models in general is a lack of domain-specific adaptability because words like “cold” can be negative (in “cold-hearted”) or neutral (in “cold drink”).

3.2 Machine Learning Models

Machine learning models for Sentiment Analysis rely on labeled datasets to train classifiers capable of predicting sentiment based on features—like word frequency, PoT tags, and syntactic dependencies—extracted from text. The labels of the dataset indicate what sentiment category a document falls into. The model learns patterns and relationships between features and sentiment labels, allowing it to classify unseen text [6]. Among the machine learning models used in SA are probabilistic classifier models, which estimate the probability of a given text belonging to a sentiment class based on its features.

An example of a probabilistic method is a Maximum Entropy Classifier, which transforms text into feature vectors and assigns each feature a weight. The model uses the weighted sum to compute the probability distribution over sentiment classes [6]. Unlike Naive Bayes classifiers, which assume conditional independence between features, Maximum Entropy models do not make these assumptions, allowing them to capture dependencies between co-occurring words—like the difference between “not good” and “very good.” This flexibility often results in more accurate sentiment predictions, especially for texts with mixed or subtle sentiment expressions [6].

An alternative approach involves using linear classifiers, which work by separating sentiment classes using a linear decision boundary within the feature space. This is often accomplished using a Support Vector Machine (SVM) model that can identify a separating hyperplane in high-dimensional spaces. Additionally, a Neural Network (NN) model can be

used to capture more complex relationships through multiple layers of processing that are necessary to find the best separation [6].

3.3 LLM-based Models

With the vast capabilities of major LLM models, they have increasingly been adapted for Sentiment Analysis tasks. Due to their ability to process and understand complex linguistic patterns, LLMs can naturally handle a wide range of sentiment-related tasks. Despite their significant potential, one of the biggest obstacles that LLM models face is in AI quality assurance, including issues of accuracy, reliability, and robustness.

A study in 2023 analyzed how a fine-tuned GPT model for sentiment analysis on Amazon reviews performed when presented with wrongly annotated reviews [5]. This study concluded that the model was demonstrated to be robust in detecting false reviews. In particular, a Distance-Based Surprise Adequacy (DSA) was developed to measure how far a given review is from correctly labeled training samples. With this additional metric, the model could flag reviews that are unusually far away from expected sentiment categories, indicating potential mislabeling or adversarial manipulation [5].

3.4 Sentiment Analysis in Social Media

A domain with many Sentiment Analysis applications is social media. Textual data coming from social media is notoriously unstructured, nuanced, and hard to generalize. Another challenge to sentiment analysis is that online users typically specify mixed feelings in posts, so it is not surprising that sentiment analysis remains a topic of ongoing interest in this domain. A 2018 study of unsupervised and supervised machine learning algorithms for sentiment classification of noisy and unusual formatted tweets and SMS messages [7]. It was concluded that features like unigrams (single words) and Part-of-Speech (POS) tags were most effective for sentence-level classification. The researchers also had to generate a robust lexicon from their training dataset to improve sentiment score assignment since tweets and messages vary heavily in writing styles [7].

A more recent study in 2023 developed a Sentiment Analysis model for Instagram text

classification using deep learning techniques that included Word Embedding Technology to represent words as vectors. This was done in a way that captures their linguistic meanings such that similar words have vectors that are close together in higher-dimensional space. The proposed model also involved a Convolutional Neural Network (CNN) to capture spatial relationships in text data with a self-attention mechanism that allows the model to weigh the importance of each word in relation to the others in the same sentence. The model the researchers developed performed well with accuracy indicators above 79% [1]. Overall, this study highlights the effectiveness of combining deep learning techniques to enhance sentiment classification, particularly in social media contexts where linguistic nuances and contextual relationships are critical for accurate analysis.

4 My Project

4.1 Overview

The goal of my project was to design and implement a personal website for reflective journaling in a structured and engaging way. Specifically, the platform allows users to write journal entries for memories associated with songs, creating a digital record of the emotions and recollections unique to music. The motivation for this software came from a common feeling that songs evoke vivid memories that might eventually fade as time passes. This website aims to preserve those associates by allowing users to write and organize journal entries, gradually building a personal archive that can be revisited over time.

To enhance functionality and user experience, this project incorporates techniques from Information Extraction (IE) and given more time, would also utilize Sentiment Analysis (SA). A key component of the software is that memories can be tagged with emotions, dates, or custom keywords. These tags make searching through memories easier and more meaningful. For example, a user might want to search for all memories with a “happy” or “College” tag. This component can be enhanced using IE and SA since suggested tags could be presented to the user after writing their memory.

4.2 Implementation

The project was implemented using the FastAPI framework for the backend and JavaScript’s React library for the frontend. PostgreSQL was used for the relational database management system to manage the database that stores all memories and associated tags.

My database consists of three tables: one for storing memories, one for storing unique tags, and one many-to-many table for associating a memory with its tags. In the tag table, there is a column indicating tag type (e.g. emotion, date, custom) for a more intuitive UI when adding and searching for tags.

FastAPI is one of the fastest Python web frameworks, leveraging asynchronous programming and efficient request handling. One of its most useful tools is its use of the Pydantic library to validate data, ensuring that incoming and outgoing data is of the correct type and form. Pydantic models were used to ensure that memories and tags included the correct elements, like a song name or tag type before sending them to the database. The *psycopg2* python library efficiently connected the backend to the database server. FastAPI was used to implement all CRUD operations needed for the website.

React is an efficient JavaScript library, as it uses a Virtual DOM to optimize rendering and updates. Using its component-based architecture, the website contains three visually appealing pages: a home page, an add-memory page, and a view-memory page. The *axios* library was utilized to make the necessary requests to the backend when adding or displaying memories.

Given the time frame of this project, the planned Sentiment Analysis components could not be effectively implemented. However, Information Extraction was incorporated into the adding a memory page of the website. When submitting a memory, the submitted text is sent through a keyword extraction model to suggest phrases and words to use when titling the memory. After some experimentation with different free models, the Sentence-BERT model showed the best results, mapping sentences to a 384-dimensional vector space. Within this model, the input memory text follows a similar process as outlined in Section 2.1, first tokenized using the CountVectorizer tool which transforms it into a vector of word counts. The tool builds a vocabulary from the input and creates a matrix where each entry

represents how many times a particular word appears in the input text. This is followed by extracting candidate keywords and passing them through the pre-trained BERT model to get its associated embedding. Then, using cosine similarity—a metric that measures the similarity of two vectors by calculating the cosine of the angle between them, quantifying how closely two vectors are aligned—between the computed candidate embedding and the document embedding, a list of top keywords is returned.

To ensure that suggested phrases are semantically meaningful, each candidate phrase is filtered using the *spaCy* library’s part-of-speech tagging system. The filtering function selects only those candidate titles that contain at least one noun, proper noun, or adjective, all of which indicate that a phrase is likely to be descriptive. These options are then displayed on the website when writing the title, facilitating the process of the user coming up with a title.

4.3 Future Work

There are two main features of the project that can be improved upon or added. First, a better title generator could be constructed using more sophisticated IE models fine-tuned for title creation. Many of these models exist, but are either not open source or require more time to fully implement and integrate into the existing website structure. Second, a Sentiment Analysis component can be implemented to extract sentiments from the memory text to suggest potential emotion tags to the user. Moreover, Named Entity Recognition and Event Extraction would be extremely useful in suggesting custom tag ideas like people, years, or events. These additions would not only improve the overall user experience but also highlight how IE and SA can be used to enhance software applications.

5 Conclusion

This paper provided an overview of current techniques, frameworks, and models used in Information Extraction (IE) and Sentiment Analysis (SA) and demonstrated their application in a personal journaling platform designed to preserve song-associated memories. Both IE and SA have proven useful in real-world scenarios, like enhancing automated article summarizing or facilitating businesses in product review analysis.

By exploring machine learning-based approaches as well as LLM applications, this work examined how methods of tokenizing, tagging, and coreference resolving facilitate in Information Extraction. Additionally, Sentiment Analysis techniques range from lexicon-based to neural network approaches to classify sentiments at the document, sentence, and entity level. The insights gained from exploring these NLP subfields were vital in the development of a web application that utilizes keyword extraction to provide memory title suggestions.

The user experience of the created website would be enhanced by future additions of more domain-specific IE systems and SA models. This project underscores the utility and versatility of IE and SA in software development and demonstrates the promising future of context-aware applications.

References

- [1] C.-P. Chan and J.-H. Yang. “Instagram Text Sentiment Analysis Combining Machine Learning and NLP”. *ACM Transactions on Asian and Low-Resource Language Information Processing*, July 2023.
- [2] H. Ji. *Information Extraction*, pages 1476–1481. Springer US, Boston, MA, 2009.
- [3] P. Mehta and S. Pandya. “A Review On Sentiment Analysis Methodologies, Practices And Applications”. *International Journal of Scientific & Technology Research*, 2020.
- [4] P. Nadkarni, L. Ohno-Machado, and W. Chapman. Natural Language Processing: An Introduction. *Journal of the American Medical Informatics Association : JAMIA*, 18:544–51, 09 2011.
- [5] T. Ouyang, H.-Q. Nguyen-Son, H. H. Nguyen, I. Echizen, and Y. Seo. “Quality Assurance of A GPT-based Sentiment Analysis System: Adversarial Review Data generation and Detection”. In *2023 30th Asia-Pacific Software Engineering Conference (APSEC)*, pages 450–457. IEEE, 2023.
- [6] S. Pandya and P. Mehta. A Review on Sentiment Analysis Methodologies, Practices and Applications. 10 2020.

- [7] J. Rout, K.-K. R. Choo, A. Dash, S. Jena, and K. Williams. “A Model for Sentiment and Emotion Analysis of Unstructured Social Media Text”. *Electronic Commerce Research*, Vol 18, March 2018.
- [8] Y. Zhu, H. Yuan, S. Wang, J. Liu, W. Liu, C. Deng, H. Chen, Z. Liu, Z. Dou, and J.-R. Wen. “Large Language Models for Information Retrieval: A Survey”. *arXiv*, 2024.